

JSP Includes, JavaBeans & Custom Tags — Assignment Answers

Topics: jsp:include, jsp:useBean, jsp:setProperty, custom JSP tag libraries (JSTL/tag files)

Q1. header.jsp + footer.jsp Included in main.jsp

header.jsp:

```
<div style="background:#1a3c6e; color:#fff; padding:12px 20px;">
  <h2 style="margin:0;">Welcome to MyWebApp!</h2>
</div>
```

footer.jsp:

```
<div style="background:#eee; color:#555; padding:10px 20px; text-align:center; font-size:12px;">
  &copy; 2026 MyWebApp. All rights reserved.
</div>
```

main.jsp — using <jsp:include>:

```
<html>
<head><title>Main Page</title></head>
<body>

  <jsp:include page="header.jsp" />

  <div style="padding:20px;">
    <p>This is the main content area of the page.</p>
  </div>

  <jsp:include page="footer.jsp" />

</body>
</html>
```

Note: <jsp:include> is a *dynamic* include — it happens at request time, so if header.jsp or footer.jsp is later changed, main.jsp automatically shows the updated version without needing to be recompiled itself.

Q2. Product.java Bean + jsp:useBean (Default Values)

```
public class Product {  
  
    private String name = "Unnamed Product";  
    private double price = 0.0;  
  
    public Product() { }    // JavaBeans require a no-arg constructor  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
  
    public double getPrice() { return price; }  
    public void setPrice(double price) { this.price = price; }  
}
```

showProduct.jsp (default values only):

```
<jsp:useBean id="product" class="Product" scope="request" />  
  
<h2>Product Details</h2>  
<p>Name: <jsp:getProperty name="product" property="name" /></p>  
<p>Price: <jsp:getProperty name="product" property="price" /></p>
```

Visiting showProduct.jsp with no query string displays the bean's default values: 'Name: Unnamed Product' and 'Price: 0.0'.

Q3. showProduct.jsp — jsp:setProperty from Request Parameters

```
<jsp:useBean id="product" class="Product" scope="request" />

<jsp:setProperty name="product" property="name" param="name" />
<jsp:setProperty name="product" property="price" param="price" />

<h2>Product Details</h2>
<p>Name: <jsp:getProperty name="product" property="name" /></p>
<p>Price: &#8377;<jsp:getProperty name="product" property="price" /></p>
```

Test URL: <http://localhost:8080/MyWebApp/showProduct.jsp?name=Earbuds&price=1999> → displays 'Name: Earbuds' and 'Price: ■1999.0'. jsp:setProperty automatically matches the param name to the bean's setter (setName/setPrice) and converts the price string to a double.

Q4. Custom JSP Tag Library — Instagram-style Follower Count ('1.5K')

Outline only, as required — no implementation code, just the process and usage.

- 1 **Create the tag handler class:** Write a Java class (e.g. `FollowerCountTag`) that extends `SimpleTagSupport` (or `TagSupport`). It would take an integer 'count' attribute, apply formatting logic (e.g. divide by 1000 and append 'K' if count $\geq$ 1000, by 1,000,000 for 'M', etc.), and write the formatted string to the page output using `getJspContext().getOut().write(...)`.
- 2 **Declare the tag in a Tag Library Descriptor (.tld file):** Create a file like `followerTags.tld` under `WEB-INF/`, defining the tag's name (e.g. 'formatCount'), the tag class, the body-content type, and its attribute ('count', type `Integer`, `required=true`).
- 3 **Package the TLD and class:** Place the compiled tag handler class under `WEB-INF/classes` and the .tld file under `WEB-INF/tlds/` (or directly under `WEB-INF/`), so the servlet container can discover it when the JSP page references it.
- 4 **Reference the tag library in the JSP page:** Add a taglib directive at the top of the JSP: `<%@ taglib uri="/WEB-INF/followerTags.tld" prefix="custom" %>`.
- 5 **Use the custom tag in the page:** Once declared, the tag can be used like a normal HTML-ish element: `<custom:formatCount count="1500" />` would output '1.5K' at that point in the rendered page, just as Instagram shows follower counts.

Example usage in a JSP page (illustrative only):

```
<%@ taglib uri="/WEB-INF/followerTags.tld" prefix="custom" %>
...
<p>Followers: <custom:formatCount count="1500" /></p>
<!-- Rendered output: Followers: 1.5K -->
```

In modern JSP development this is often done more simply using JSTL's `functions/fmt` tags or a small EL function, but writing a custom tag as above is the classic approach when the container doesn't already provide the exact formatting needed.